

CONSTRUIR UN CONTRATO EN VIVO, ROMPIÉNDOLO SEIS VECES

Este guion va al lado del teclado. No es material para proyectar.

La idea rectora: **cada versión del contrato se rompe en vivo y la función siguiente existe para arreglar eso**. Ninguna función entra porque el temario lo diga.

DURACIÓN	DONDE SE ESCRIBE	CÓDIGO	PLAN B DE LA MORA
180 min (minutaje en §2)	Remix app.remix.live	pasos/paso-1.sol ... paso-6.sol	25 pruebas en Hardhat

CÓMO USAR ESTE GUION

Cada paso tiene una ficha con seis partes, siempre en el mismo orden: **qué se escribe** (remite al archivo, no repite el código), **dígalo así** (la frase exacta con la que se presenta la idea), **la demostración clic por clic** en Remix, **la rotura** que usted provoca a propósito, **la pregunta** al grupo con su respuesta esperada, y **el recorte** si se está quedando sin tiempo.

Las frases de «dígalo así» están escritas para leerse casi literal. No son el resumen: son el gancho. Si solo alcanza a decir una cosa por paso, diga esa.

Los seis archivos compilan limpios con **0.8.28**, sin avisos. Cada uno termina con un bloque de comentario **EN CLASE** que es la columna vertebral de este guion: si pierde el hilo, baje al final del archivo abierto.

01 Antes de entrar al salón

15 MIN DE MONTAJE

Programar en vivo falla por el montaje, no por el código. Esta lista es corta a propósito.

QUÉ	CÓMO SE COMPRUEBA QUE QUEDÓ
1 Remix abierto en <code>app.remix.live</code> , con el panel <i>Solidity Compiler</i> ya puesto en 0.8.28+commit.7893614a .	El panel muestra esa versión exacta, no una más nueva. Es la versión congelada del semestre.
2 Los seis <code>.sol</code> abiertos en su editor local, en otra ventana, detrás de Remix.	Son su chuleta. Nunca se proyectan: si los estudiantes ven el paso 6, se pierde el efecto.
3 Un archivo nuevo y vacío en el explorador de archivos de Remix, llamado <code>PrestamoEquipos.sol</code> .	Ahí se escribe todo. Los seis pasos son el mismo archivo que crece: no se crean seis archivos.

4	ENVIRONMENT en Remix VM . En las versiones recientes hay dos desplegables : el entorno (<i>Remix VM</i>) y, al lado, el <i>hard fork</i> (por ejemplo <i>Osaka</i>). Cualquier fork sirve para esta clase. Y Value en 0 antes de desplegar.	El selector <i>Account</i> lista 10 cuentas con 100 ETH cada una. Si no las ve, no está en la Remix VM.
5	Plan A de la mora : una terminal con <code>npx hardhat node</code> corriendo en <code>material/laboratorios-evm</code> , y una segunda terminal libre en la misma carpeta.	La primera terminal queda imprimiendo las cuentas de prueba y dice que escucha en <code>127.0.0.1:8545</code> .
6	Plan B de la mora : una tercera terminal, ya ubicada en <code>material/laboratorios-evm</code> , lista para lanzar las pruebas.	Se usa si la red local no conecta con Remix. Ver §11.
7	Tamaño de letra del editor de Remix subido, y la terminal de Remix visible sin tener que desplazarse.	La mitad de la clase es leer el error . Si no se lee desde la última fila, no sirve.
8	En el tablero, escritas estas seis palabras y nada más: RECORDAR · DECIR NO · MUCHOS · QUIÉN MANDA · HASTA CUÁNDO · QUE CUESTE	Es la agenda. Se va tachando. Los estudiantes ven el avance sin ver el código que falta.

LA REGLA DE ORO DE LA CLASE

Usted **no** escribe las seis versiones y luego explica. Escribe una versión corta, la ejecuta, la rompe delante de ellos, y recién entonces escribe lo que arregla la rotura. Si en algún momento se encuentra explicando una palabra de Solidity que todavía no hizo falta, deténgase: ese es exactamente el error de la clase anterior.

02 Minutaje · 180 minutos

SUMA EXACTA

MINUTO	DURA	BLOQUE	QUÉ ENTRA Y POR QUÉ
00 – 15	15	Apertura	Reconocer que la explicación anterior quedó enredada. Plantear el problema real del laboratorio de equipos. Decir la regla de la clase.
15 – 30	15	Paso 1 · Recordar	Estado y <code>msg.sender</code> . Línea base. Se rompe: cualquiera se lleva un equipo ya prestado.
30 – 50	20	Paso 2 · Decir no	<code>revert</code> con errores personalizados y <code>view</code> . Se rompe: solo existe un equipo.
50 – 75	25	Paso 3 · Muchos	<code>struct</code> , <code>mapping</code> , <code>calldata</code> . Se rompe: cualquiera registra inventario.

75 – 85	10	Pausa	Cortar aquí y no en otro punto: el paso 3 cierra con una pregunta abierta que trabaja sola durante la pausa.
85 – 105	20	Paso 4 · Quién manda	<code>constructor</code> , <code>modifier</code> , dueño. Se rompe: nadie devuelve nunca.
105 – 130	25	Paso 5 · Hasta cuándo	<code>block.timestamp</code> , plazo, mora, división entera. Se rompe: estar en mora no cuesta nada. Aquí entra el reloj de la red local.
130 – 160	30	Paso 6 · Que cueste	<code>payable</code> , depósito, patrón de retiro, eventos, <code>receive / fallback</code> . Es el paso más largo y el que no se recorta.
160 – 180	20	Cierre	Recorrido de las seis roturas y la pregunta del curso: ¿esto necesitaba una blockchain?

SI VA ATRASADO · EL ORDEN EN QUE SE SACRIFICA

Primero se recorta **dentro** de cada paso (cada ficha dice qué). Si eso no alcanza, el orden de sacrificio es: **1.** la demostración de `recuperar()` en el paso 4; **2.** la demostración en vivo de la mora en el paso 5, que pasa a plan B (proyectar las pruebas, 3 minutos); **3.** el envío directo de ether del paso 6. **Nunca** se recorta el cierre: la pregunta del curso es lo único que no pueden sacar de un tutorial.

Los primeros tres minutos deciden si la clase tiene credibilidad. Reconocer que la explicación anterior quedó enredada **suma**: muestra que usted sabe distinguir una explicación que funcionó de una que no, que es justo lo que se les está pidiendo aprender. Lo que resta credibilidad es seguir como si nada.

DÍGALO ASÍ · ARRANQUE, MIN 00

«La clase pasada les expliqué Solidity al revés y se me enredó. Les di `struct`, `mapping`, `modifier` y `payable` como si fueran vocabulario para memorizar, y el vocabulario sin un problema que resolver no se sostiene. La culpa es del orden, no de ustedes.»

«Hoy lo hacemos al contrario. Vamos a escribir un contrato que no sirve, lo vamos a romper en vivo, y cada cosa nueva va a entrar solo cuando ustedes ya vieron qué se rompe sin ella. Al final del día, si una función del contrato no tiene una rotura que la justifique, sobra, y la borramos.»

«Van a ver seis versiones. Las seis compilan. Las cinco primeras están mal.»

El problema, antes de una sola línea de código

Plantéelo sin mencionar blockchain. Que lo reconozcan como un problema de su universidad.

DÍGALO ASÍ · EL PROBLEMA, MIN 04

«El laboratorio presta equipos: proyectores, kits de Arduino, cámaras. El registro vive en una hoja de papel pegada a la pared. Nadie sabe quién tiene el proyector. Nadie sabe desde cuándo. Y cuando no vuelve, no pasa nada.»

«Eso es todo lo que vamos a resolver hoy. Vamos a terminar con un contrato de 190 líneas que lo resuelve. Y en los últimos veinte minutos voy a sostener que, para nuestra universidad, este contrato **no** hacía falta. Las dos cosas son verdad y saber distinguirlas es el curso.»

Lo único de logística que se dice ahora

- Hoy **miran** y preguntan. No corren detrás copiando. La guía del estudiante trae los seis pasos para repetirlo en casa, con la salida esperada de cada comprobación.
- Todo pasa en la **Remix VM**: una cadena de juguete dentro del navegador, con diez cuentas y 100 ETH de mentira cada una. No hay dinero real en ningún momento de esta clase.
- Si algo no compila, **no lo arreglo yo solo**: leemos el error juntos. Leer errores es la mitad del oficio.

SI ALGUIEN PREGUNTA «¿Y ESTO VA EN EL PARCIAL?»

Respuesta corta y honesta: «Lo que va es saber por qué cada función está ahí. Si en el parcial le pido `mapping` de memoria, el parcial está mal hecho.»

1

QUÉ SE ESCRIBE · `pasos/paso-1.sol` · 15 LÍNEAS ÚTILES

Dos variables de estado, dos funciones de tres palabras

Escríbalos a mano, completo, delante de ellos. Son quince líneas: alcanza y es la última vez en la clase que se puede escribir todo desde cero. Lo único que no se escribe a mano son los comentarios del archivo.

Las cuatro cosas que se nombran al escribir, y ninguna más: `pragma solidity 0.8.28;` (la versión, fija); `contract`; las dos variables de estado; y `msg.sender`. Nada de tipos de datos, nada de visibilidad más allá de «`public` me regala la función de lectura gratis».

DÍGALO ASÍ · AL ESCRIBIR `prestar()`

«Fíjense en lo que esta función **no** recibe: no recibe quién la llama. No hace falta. En Solidity nadie dice quién es: lo demuestra firmando. `msg.sender` es la dirección que firmó esta transacción, y no se puede falsificar sin la llave privada. Eso que en cualquier otro programa es un campo de formulario que uno puede mentir, aquí viene de la firma.»

Y al escribir `devolver()`: «`address(0)` es la dirección cero, el vacío de Ethereum. La usamos para decir *no lo tiene nadie*. Acuérdense de esta dirección, va a volver todo el día.»

Demostración en Remix · clic por clic

QUÉ HACE	QUÉ DEBE APARECER
1 <i>Solidity Compiler</i> → compilar.	Marca verde en el ícono del compilador. Cero avisos. Dívalo en voz alta: «cero avisos, eso es parte del trabajo».
2 <i>Deploy & Run Transactions</i> → <i>ENVIRONMENT</i> : primera opción Remix VM → <i>Account</i> : la primera cuenta → Deploy .	La terminal de Remix registra la transacción de despliegue. En <i>Deployed Contracts</i> aparece <code>PRESTAMO EQUIPOS AT 0x...</code> . Expandirlo con el triángulito de la izquierda.
3 Botón <code>prestadoA</code> (azul: es una lectura, no cuesta nada).	<code>0x00</code> . «Nadie lo tiene. Y fíjense que no tuve que inicializar nada: en Solidity todo arranca en cero.»
4 Botón <code>prestar</code> (naranja: es una transacción). Cuenta 1.	Transacción correcta en la terminal. Volver a leer <code>prestadoA</code> : ahora es la dirección de la cuenta 1. Compárela con el selector <i>Account</i> carácter por carácter: los primeros cuatro y los últimos cuatro.
5 Cambiar <i>Account</i> a la cuenta 2. Botón <code>prestar</code> otra vez.	La transacción sale bien. <code>prestadoA</code> ahora es la cuenta 2.

LA ROTURA · Y ES LA MÁS IMPORTANTE DE LA CLASE

No hay error. No hay nada rojo. La transacción del paso 5 sale **verde**. Deténgase ahí tres segundos en silencio y después dígalos:

«El proyector cambió de manos y la cuenta 1 nunca lo devolvió. No hay ningún mensaje de error porque el programa hizo exactamente lo que le pedí. La rotura no es un error rojo: es que todo salió bien y el resultado es imposible en la vida real. Eso es el 90 % de los errores en contratos: no son excepciones, son reglas que nadie escribió.»

Pregunta al grupo

Se pregunta: «El código no tiene un solo error de sintaxis y acaba de hacer algo imposible. ¿Dónde está el error, entonces?»

Respuesta esperada: falta una validación; el contrato nunca comprueba si el equipo ya está prestado. **Si nadie contesta**, reformule hacia la acción: «¿qué tendría que haber pasado cuando la cuenta 2 apretó el botón?» — que la transacción se cayera. Esa es la palabra que usted quiere escuchar antes de pasar al paso 2.

Si alguien dice «pues que lo valide la aplicación»: excelente, y es la respuesta equivocada más útil del día. Guárdela: «si la validación vive en la aplicación, cualquiera que llame al contrato sin pasar por la aplicación se la salta. Y aquí llamar al contrato directo es gratis y nadie lo puede impedir. En el paso 2 van a ver que yo lo llamo directo desde Remix, sin aplicación ninguna.»

SI VA SIN TIEMPO No muestre `devolver()` en este paso: entra solo en el paso 2, donde además necesita una validación. Con desplegar, leer `prestadoA` y llamar `prestar()` dos veces desde dos cuentas, el paso 1 ya cumplió.

2

QUÉ SE ESCRIBE · `pasos/paso-2.sol`Dos errores personalizados, una función `view` y dos guardias

Sobre el mismo archivo. Se agregan, en este orden: las dos declaraciones `error`, la función `estaDisponible()` marcada `view`, y una línea de guardia al principio de `prestar()` y otra al de `devolver()`.

OJO CON EL VOCABULARIO · DIFERENCIA CON LA CLASE ANTERIOR

Este contrato valida con `if (...) revert MiError(...);`, **no** con `require`. Es deliberado: el error con nombre propio y parámetros es lo que se usa hoy, y `require(condición, "texto")` aparece **una sola vez en toda la clase**, en el paso 6, donde se explica la diferencia con el ejemplo al lado. Si usted dice «require» aquí por costumbre, corrija en voz alta: les enseña que las dos formas existen y cuál se prefiere.

DÍGALO ASÍ · AL DECLARAR LOS ERRORES

«Un contrato que no sabe decir NO no sirve para nada. Pero fíjense cómo dice no:

`EquipoOcupado(address loTiene)`. Eso no es un mensaje de error, es un **dato**. El error no solo dice que no se puede: dice *quién* lo tiene. Un error con nombre propio y con parámetros cuesta menos gas que un texto y una interfaz lo puede leer y mostrar bonito. Un `"error: no se puede"` no le sirve a nadie.»

DÍGALO ASÍ · AL MARCAR `view`

«`view` quiere decir: esta función lee el estado y no lo cambia. Y eso tiene una consecuencia que vale dinero: llamarla desde fuera **no genera una transacción**, no se mina, no cuesta gas. Es una pregunta, no una orden. Por eso Remix le pinta el botón de otro color: esos botones son gratis, los otros no.»

Demostración en Remix · clic por clic

Recompilar y **volver a desplegar**. Dígalos: «cada *Deploy* crea un contrato nuevo y vacío; el de hace un rato sigue ahí con su estado, pero ya no nos sirve. Usamos siempre el último de la lista.»

CUENTA · FUNCIÓN ·
ARGUMENTOS

QUÉ DEBE APARECER

1 Cuenta 1 → `estaDisponible()` `true`, debajo del botón. Gratis.

2 Cuenta 1 → `prestar()` Funciona. `estaDisponible()` ahora da `false`.

3 Cuenta 2 → `prestar()`

REVIERTE. La terminal marca la transacción como fallida y muestra el nombre del error con su argumento: `EquipoOcupado` y la dirección de la cuenta 1. Expanda el detalle de la transacción en la terminal y **lea esa dirección en voz alta.**

4 Cuenta 2 → `devolver()`

REVIERTE con `NoLoTienesTu`. «Esta es la mala de verdad: sin esta línea, yo devuelvo el equipo de otro y me lo llevo al segundo siguiente.»

5 Cuenta 1 → `devolver()`, y
después cuenta 2 →
`prestar()`

Las dos funcionan. El ciclo completo cierra.

EL DETALLE QUE VALE LA PENA SEÑALAR AQUÍ

Cuando una transacción revierte, **no queda nada a medias**. No es que se haya guardado la mitad. Dígalo: «revertir no es *fallar*: es deshacer todo lo que esta transacción tocó, como si nunca hubiera pasado. Lo único que se pierde es el gas. En una base de datos normal, esto se llama transacción y hay que pedirlo; acá es lo único que hay.»

LA ROTURA · QUE LA DIGAN ELLOS

No hay nada que romper con un clic: el contrato ya se defiende bien. La rotura es de **diseño** y se provoca con una pregunta. Dígala tal cual:

«Listo, el proyector está bajo control. El laboratorio tiene cuarenta equipos. ¿Vamos a desplegar cuarenta contratos?»

Pregunta al grupo

Se pregunta: esa misma. **Respuesta esperada:** no; hace falta que un solo contrato guarde muchos equipos, cada uno con su identificador y su estado.

Si contestan «sí, cuarenta contratos» —pasa, y no es una respuesta tonta— empújela hasta el absurdo con datos: «cuarenta despliegues, cuarenta direcciones que alguien tiene que anotar, y cuando entre un equipo nuevo, otro despliegue. ¿Y para saber qué hay disponible? Cuarenta preguntas.» Ahí cae solo.

SI VA SIN TIEMPO Salte la fila 4 de la tabla (`devolver()` desde la cuenta que no lo tiene). Es la validación más bonita del paso, pero la rotura que abre el paso 3 no depende de ella. Lo que **no** se salta es la fila 3: ver el error personalizado con su dirección adentro es el punto del paso.

3

QUÉ SE ESCRIBE · `pasos/paso-3.sol``struct` , `mapping` , un contador, `calldata` y un campo `existe`

Es el paso con más cambios: las dos variables de estado se van y entra el `struct Equipo` , el `mapping` , `totalEquipos` , la función `registrar()` , y todas las funciones pasan a recibir un `uint256 id` . Reescribir las firmas en vivo es parte del valor: ven que el tipo del parámetro se propaga a todo.

DÍGALO ASÍ · AL ESCRIBIR EL `struct` Y EL `mapping`

«Un `struct` es un tipo que usted inventa, para que los datos que van juntos viajen juntos. Un `mapping` es una tabla: usted le da una llave, le devuelve un valor. Hasta ahí es un diccionario de cualquier lenguaje.»

«Pero trae dos rarezas que les van a arruinar un proyecto si no las saben. **Uno:** no se puede recorrer. No existe *dame todos los equipos*. Si usted necesita la lista, la lleva aparte, a mano. **Dos:** toda llave existe desde siempre, llena de ceros. Yo le puedo preguntar por el equipo 7.000.000 y me va a contestar, sin quejarse, con un equipo en blanco.»

Y por eso, inmediatamente, el campo `existe` : «esta línea de aquí es la que distingue *el equipo 99 no está registrado* de *el equipo 99 está libre*. Sin ella el contrato contesta una mentira muy cómoda.»

DÍGALO ASÍ · AL PONER `calldata` Y AL ARRANCAR LOS `id` EN 1

«`calldata` : el texto se lee directo de los datos de la transacción, sin copiarlo a memoria. Es la opción más barata para un parámetro que la función no va a modificar. Es una de esas palabras que en otro lenguaje no existiría, y existe acá porque cada copia cuesta.»

«Y fíjense en el truco: `totalEquipos++` va **antes** de asignar el `id` , para que el primer equipo sea el 1 y no el 0. Así *id cero* significa siempre *no existe*. En un lenguaje donde todo arranca en cero por defecto, reservarse el cero para decir *nada* ahorra errores.»

Demostración en Remix · clic por clic

	CUENTA · FUNCIÓN · ARGUMENTOS	QUÉ DEBE APARECER
1	Cuenta 1 → <code>registrar</code> con <code>"Proyector Epson"</code> (con comillas, es un <code>string</code>)	Transacción correcta. Advertencia: <code>registrar</code> devuelve el <code>id</code> , pero al ser una transacción Remix no muestra el valor devuelto en el botón. Es un punto excelente: «el valor de retorno de una transacción no se lo puede quedar nadie desde fuera; para eso están los eventos, que llegan en el paso 6».
2	Cuenta 1 → <code>registrar</code> con <code>"Kit Arduino"</code>	Correcta. Leer <code>totalEquipos</code> : <code>2</code> .

3	<code>equipos</code> con <code>1</code> , y después con <code>2</code>	Los tres campos del <code>struct</code> : <code>nombre</code> , <code>prestadoA</code> en la dirección cero, <code>existe: true</code> . «Y esta función de lectura no la escribí: me la regaló el <code>public</code> del <code>mapping</code> .»
4	Cuenta 1 → <code>prestar</code> con <code>1</code> . Cuenta 2 → <code>prestar</code> con <code>2</code> .	Las dos funcionan y conviven. Leer <code>equipos(1)</code> y <code>equipos(2)</code> : dos dueños distintos, un solo contrato.
5	Cuenta 2 → <code>prestar</code> con <code>1</code>	REVIERTE con <code>EquipoOcupado</code> y la cuenta 1 adentro. La defensa del paso 2 sigue viva con cuarenta equipos: se escribió una sola vez.
6	<code>estaDisponible</code> con <code>99</code>	REVIERTE con <code>EquipoNoRegistrado(99)</code> . «Fíjense en algo raro: esta función es <code>view</code> , es una simple pregunta, y también puede revertir. Y revierte a propósito: sin el campo <code>existe</code> me habría contestado <i>sí, disponible</i> .»
7	Cuenta 1 → <code>prestar</code> con <code>99</code>	REVIERTE también con <code>EquipoNoRegistrado(99)</code> , no con <code>EquipoOcupado</code> . « <code>prestar</code> llama a <code>estaDisponible</code> , y el error de adentro sale hasta afuera tal cual. Los <code>revert</code> se propagan.»

LA ROTURA · HÁGALA CON LA CUENTA 3, Y ANÚNCIELA

Cambie *Account* a la **cuenta 3** y diga: «esta cuenta es un estudiante cualquiera. No es el laboratorio, no es nadie.» Llame a `registrar` con `"Portatil que no existe"`.

Funciona. Lea `totalEquipos`: `3`. «Acabo de meter inventario inventado en el registro del laboratorio desde una cuenta cualquiera, y el contrato me dio las gracias. Mañana meto cuatrocientos y el registro no vale nada.»

Pregunta al grupo

Se pregunta: «¿Quién debería poder registrar equipos? Y la segunda, que es la difícil: ¿cómo hace el contrato para saber que yo soy esa persona?»

Respuesta esperada: solo el laboratorio o su encargado; y el contrato lo sabe porque guarda una dirección privilegiada y la compara con `msg.sender`. La segunda mitad es la que importa: ya tienen la herramienta del paso 1 y no la habían vuelto a usar.

Si alguien dice «con un usuario y una contraseña»: es la pregunta de oro. «En la cadena no hay usuarios ni contraseñas, y no puede haberlos: todo lo que el contrato guarda es público, incluida la contraseña. Lo único que hay es: quién firmó. Guardamos una dirección y los comparamos.»

SI VA SIN TIEMPO Junte las filas 1 y 2 registrando un solo equipo y salte la fila 7. Lo que **no** se salta es la fila 6 (`estaDisponible(99)`): el campo `existe` es lo más fácil de olvidar y lo que más errores silenciosos causa en los proyectos de ellos.

Corte exactamente aquí, con la pregunta de «quién debería poder registrar» sin responder. Trabaja sola durante diez minutos.

Antes de salir del salón: no cierre ni recargue la pestaña de Remix. Según la configuración, al recargar el navegador la Remix VM puede volver a su estado inicial y perdería el inventario registrado. Si va a tocar el computador, anote primero en el tablero el `id` de cada equipo registrado: reconstruirlo cuesta treinta segundos y se hace mientras habla.

Al retomar, min 85, dígalos así: «Antes de la pausa un estudiante cualquiera metió un portátil inventado en el inventario del laboratorio. Vamos a arreglar eso, y arreglarlo va a abrir el problema más incómodo del día.»

4

QUÉ SE ESCRIBE • `pasos/paso-4.sol``constructor`, `modifier soloEncargado`, la variable `encargado` y `recuperar()`

Cuatro cosas, en este orden: la variable `address public encargado`, el `constructor` que la graba, el `modifier`, y pegarle `soloEncargado` a `registrar`. La función `recuperar()` entra al final y tiene su propia justificación (abajo).

DÍGALO ASÍ • AL ESCRIBIR EL CONSTRUCTOR

«El `constructor` corre **una sola vez en la vida del contrato**: en el momento del despliegue. Nunca más. No se puede volver a llamar, no aparece como botón, no existe después. Lo que se grabe ahí queda grabado. Y lo que estoy grabando es quién soy yo, ahora, desplegando esto.»

DÍGALO ASÍ • AL ESCRIBIR EL MODIFICADOR

«Un `modifier` es una condición que se repite, escrita una sola vez. Y el `_;` del final no es un adorno: es el **hueco donde se mete el cuerpo de la función**. Todo lo que está antes del `_;` corre antes de la función; si pusiera algo después, correría después. Por eso el orden importa: la validación va arriba, siempre.»

Al pegarle el modificador a `registrar`, hágalo despacio y señale la palabra: «mire dónde va: entre la visibilidad y el `returns`. Una palabra, y la función queda cerrada para todo el mundo menos para una dirección.»

Demostración en Remix • clic por clic

CUENTA • FUNCIÓN • ARGUMENTOS		QUÉ DEBE APARECER
1	Desplegar con la cuenta 1 . Leer <code>encargado</code> .	La dirección de la cuenta 1. «Nadie me preguntó nada. Lo decidió el hecho de que fui yo quien desplegó.»
2	Cuenta 3 → <code>registrar</code> con <code>"Portatil que no existe"</code>	REVIERTE con <code>SoloElEncargado</code> y, adentro, la dirección de la cuenta 3. «El mismo clic que funcionó hace quince minutos ahora se cae, y el error me dice quién lo intentó.»
3	Cuenta 1 → <code>registrar</code> con <code>"Proyector Epson"</code>	Funciona. <code>totalEquipos: 1</code> .
4	Cuenta 2 → <code>prestar</code> con <code>1</code>	Funciona. <code>prestar</code> no lleva el modificador, a propósito: prestar lo puede hacer cualquiera, registrar no. «Decidir qué función lleva la guardia es el diseño; poner la guardia es lo fácil.»
5	Cuenta 1 → <code>recuperar</code> con <code>1</code>	<code>equipos(1)</code> vuelve a <code>prestadoA</code> en cero, sin que la cuenta 2 haya hecho nada.
6	Cuenta 2 → <code>recuperar</code> con <code>1</code>	REVIERTE con <code>SoloElEncargado</code> .

Antes de la fila 5, justifique `recuperar()` sin tecnicismos: «alguien se graduó, perdió la billetera, o simplemente no devuelve. Sin esta función ese equipo queda prestado para siempre, en un registro que nadie puede corregir. Es la primera función que entra no porque algo se rompió, sino porque algo **se puede trancar para siempre**, y en la cadena *para siempre* es literal.»

LA ROTURA · NO ES TÉCNICA, Y ES LA CONVERSACIÓN QUE MÁS VALE DEL DÍA

No hay nada que clicar. Hay algo que confesar. Dígalo mirando al grupo, no a la pantalla:

«Acabamos de arreglar el problema y de crear uno más grande. Le dimos a **una sola dirección** el poder de quitarle el equipo a cualquiera. Y miren el contrato: **no hay forma de cambiar de encargado**. Si esa llave se pierde, nadie registra un equipo nunca más y nadie recupera nada, jamás. Si se la roban, el inventario es del ladrón. Y no puedo arreglarlo editando el contrato, porque ya está desplegado.»

«Lo grave no es tener un dueño. Casi todo contrato tiene uno. Lo grave es no darse cuenta de que lo hay. Cuando lean un contrato ajeno, la primera pregunta es: *¿quién manda aquí, y qué pasa si esa llave se pierde?*»

Pregunta al grupo

Se pregunta: «¿Qué pasa en este contrato, exactamente, si el encargado se gradúa y bota el computador con la billetera?»

Respuesta esperada: el inventario queda congelado para siempre; no se registra ni se recupera nada más; y no hay nada que hacer, porque el contrato no tiene cómo cambiar de encargado. **Remate:** «lo que falta es poder traspasar el cargo. Eso ya está escrito y auditado: se llama `Ownable`, de OpenZeppelin, y es este mismo código con transferencia de propiedad. Lo vemos en la próxima sesión. Hoy lo escribimos a mano para que cuando lo usen sepan qué hay adentro.»

SI VA SIN TIEMPO Salte las filas 5 y 6 (`recuperar`) y escriba la función sin demostrarla, diciendo en una línea para qué está. Lo que **no** se recorta nunca es la rotura de este paso: es la que sostiene la mitad del curso.

5

QUÉ SE ESCRIBE · `pasos/paso-5.sol`

```
uint64 vence en el struct, PLAZO_MAXIMO, prestar(id, dias), estaVencido,
diasDeMora
```

El `struct` gana el campo `vence`; `prestar` gana el parámetro `dias` y tres líneas; entran la constante y las dos consultas nuevas. `devolver` y `recuperar` tienen que poner `vence` en cero: señálelo, es el típico olvido que deja basura en el estado.

DÍGALO ASÍ · AL ESCRIBIR `block.timestamp`

«El contrato no tiene reloj. No puede preguntar la hora, no puede llamar a un servidor, no puede esperar. Lo único que sabe del tiempo es `block.timestamp`: la hora que el bloque dice que es, en segundos, sin decimales.»

«Y dos avisos que van siempre juntos con esa línea. **Uno:** quien produce el bloque la puede mover unos segundos. Para *catorce días* da exactamente igual; para sortear una rifa o decidir quién gana, **jamás:** es dinero regalado al que arma el bloque. **Dos:** es un `uint256` y nuestro campo es `uint64`, por eso el `uint64(...)`. Pero la suma de al lado no la protejo yo: si desbordara, Solidity 0.8 revierte sola. Eso antes de 2021 había que escribirlo a mano y ahí se perdió mucha plata.»

DÍGALO ASÍ · AL ESCRIBIR `diasDeMora` (LA LÍNEA DE LA DIVISIÓN)

«Acá hay algo que descoloca a todo el mundo la primera vez: **en Solidity no hay decimales**. No hay `float`, no hay coma. Esta división trunca: un día y veintidós horas de mora son *un* día. Si el día de mora costara mil pesos, el contrato le está regalando casi mil. No es un error del lenguaje: es una decisión que usted tiene que tomar a propósito, y escribirla.»

El comentario de gas del archivo vale leerlo en voz alta. «`address` son 20 bytes, `uint64` ocho, `bool` uno: 29, y caben en una ranura de 32. Si `vence` fuera `uint256`, harían falta dos ranuras y cada préstamo costaría más. Ordenar los campos de un `struct` es de las pocas optimizaciones que valen la pena pensar desde el principio.»

Demostración en Remix · clic por clic

CUENTA · FUNCIÓN · ARGUMENTOS		QUÉ DEBE APARECER
1	Desplegar con cuenta 1 → <code>registrar</code> "Proyector Epson". Leer <code>PLAZO_MAXIMO</code> .	1209600. Pregunte cuánto es: catorce días en segundos. «Escribí 14 <code>days</code> y el compilador lo convirtió. <code>days</code> , <code>hours</code> , <code>weeks</code> existen y son solo multiplicaciones.»
2	Cuenta 2 → <code>prestar</code> con 1 y 2	Funciona. Leer <code>equipos(1)</code> : <code>vence</code> es un número de diez cifras. «Ese es el reloj de Unix: segundos desde el 1.º de enero de 1970. No es un adorno, es el único formato de fecha que hay acá.»

3 `estaVencido` con `1` . `false` y `0` .
`diasDeMora` con `1`

4 Cuenta 1 → `devolver` con `1` . El `devolver` revierte (`NoLoTienesTu` : es de la cuenta 2). El `prestar` con cero días **REVIERTE** con `PlazoInvalido(1209600)` : el error no dice «mal», dice cuál es el máximo.

5 Cuenta 2 → `prestar` con `1` y `15` ; y después con `1` y `14` El de 15 revierte con `PlazoInvalido` ; el de 14 **entra**. «El borde exacto sí vale. Los bordes son donde viven los errores, y hay una prueba escrita solo para este.»

LA ROTURA · Y AQUÍ LA ROTURA ES DE LA HERRAMIENTA, NO DEL CONTRATO

Intente mostrar la mora en la Remix VM y fracase **a propósito y en voz alta**. Dígalo así:

«Quiero mostrarles la mora. Presté el equipo por un día. Entonces espero un día... y no pasa nada, porque en la máquina virtual de Remix el reloj no avanza solo, y en una red de prueba pública como Sepolia tampoco lo puedo adelantar: el tiempo es el que es. No hay botón de *mañana*.»

«Esto no es un detalle: es la razón por la que todo contrato con plazos **se prueba en una red local propia**. Es la única donde yo mando el reloj. Vean.»

Cómo se demuestra la mora · plan A y plan B

Las dos formas están verificadas y las dos sirven. Decida en el minuto 118 según cómo vaya el tiempo y si la terminal del montaje (§1, punto 5) está viva.

PLAN	QUÉ SE HACE	QUÉ DEBE APARECER
A 6 MIN	1. En Remix, <i>ENVIRONMENT</i> → el entorno de desarrollo que conecta con un nodo de Hardhat (en la documentación, Hardhat Provider; en el desplegable puede aparecer dentro del grupo <i>Dev</i>, como <i>Dev - Hardhat Provider</i>). El nodo de <code>npx hardhat node</code> ya tiene que estar corriendo. 2. Volver a desplegar ahí y prestar el equipo 1 por <code>1</code> día. 3. En la segunda terminal: <pre>\$env:DIAS="3"; npx hardhat run scripts/demo/adelantar-reloj.js -network localhost</pre> 4. En Remix, llamar otra vez <code>estaVencido(1)</code> y <code>diasDeMora(1)</code> .	La terminal imprime el bloque y la fecha antes, el bloque y la fecha después, y <code>el reloj avanzó 3.00 días (259200 segundos)</code> . En Remix: <code>estaVencido</code> pasa de <code>false</code> a <code>true</code> y <code>diasDeMora</code> de <code>0</code> a <code>2</code> (tres días transcurridos menos un día de plazo). «Yo acabo de mover el tiempo. En una cadena pública eso no se puede, y es exactamente por eso que una cadena pública sirve para algo.»

B
3 MIN

No necesita Remix ni la red local. En la tercera terminal, en

```
material/laboratorios-evm:  
npx hardhat test  
test/demo/PrestamoEquipos.test.js
```

Proyectar la corrida y detenerse en el bloque *mora*.

25 pruebas en verde. Lea en pantalla los nombres de cuatro: que pasado el plazo `estaVencido` es cierto y `diasDeMora` cuenta 3; que 1,9 días de mora se cuentan como 1 (la división entera, ahora medida); que devolver en mora deja el depósito para el laboratorio; y que el último segundo del plazo cuenta como a tiempo. «Esto es lo que hace una prueba y una demostración no: mueve el tiempo, y lo vuelve a mover igual mañana.»

SI ELIGE EL PLAN A · LO QUE SE DAÑA Y CÓMO SE VUELVE

Al cambiar de entorno, el contrato de la Remix VM **no** viaja: hay que volver a desplegar y volver a registrar. Son dos transacciones, treinta segundos. Datos de la red local, por si hace falta ponerlos a mano: **chainId 31337**, RPC **http://127.0.0.1:8545**. Para el paso 6 puede quedarse en la red local (el depósito funciona igual) o volver a la Remix VM; si vuelve, avise que el estado del paso 5 se queda atrás.

Pregunta al grupo

Se pregunta: «`estaVencido(1)` dice `true`. El contrato sabe que ese equipo está en mora. ¿Y ahora qué? ¿Qué hace con esa información?»

Respuesta esperada: nada. No puede ir a buscar el equipo, no puede llamar a nadie, no puede bloquear un carné. Deje que el silencio incomode y después remate: «un contrato no tiene manos y no tiene teléfono. Lo único que puede hacer en el mundo es **mover plata**. Entonces, si estar en mora tiene que costar algo, tiene que costar plata, y la plata tiene que estar adentro antes. Eso es el paso 6.»

SI VA SIN TIEMPO Vaya directo al plan B: ahorra tres minutos y no hay nada que montar. Salte la fila 5 de la tabla (los bordes del plazo) y menciónela al pasar. Lo que **no** se salta es la rotura: que el reloj no se pueda adelantar en Remix hay que decirlo explícitamente, porque si no, los estudiantes van a pensar en casa que su contrato está mal.

6

QUÉ SE ESCRIBE · `pasos/paso-6.sol`

`payable`, `immutable deposito`, `mapping saldo`, `retirar()`, tres eventos, `receive` y `fallback`

Es el paso más largo. Escríbalo en cuatro tramos y ejecute entre cada uno: **(a)** `deposito` `immutable` + el constructor con parámetro + `payable` y el `require` del monto en `prestar`; **(b)** el `mapping saldo` y las tres líneas de `devolver`; **(c)** `retirar()`; **(d)** los eventos, y `receive` / `fallback` al final.

DÍGALO ASÍ · AL ABRIR EL PASO, ANTES DE ESCRIBIR NADA

«Hasta aquí el contrato sabía quién está en mora y no podía hacer absolutamente nada. Y no es que le falte código: es que un contrato no tiene manos. No va a ir por el proyector.»

«Lo único que un contrato puede hacer en el mundo real es **tener plata y decidir si la devuelve**. Eso es todo. Y eso es exactamente lo único que lo hace distinto de un formulario de Google con una base de datos: en el formulario, la regla la aplica una persona que puede decidir no aplicarla. Acá la regla y la plata viven en el mismo sitio, y la regla se cumple sola. Si entienden esta frase, entienden para qué sirve esta tecnología.»

DÍGALO ASÍ · AL ESCRIBIR `payable` Y EL MONTO EXACTO

«`payable` es un interruptor: sin esa palabra, la función **rechaza** cualquier ether que le manden. No es que lo ignore: revierte. Y `msg.value` es cuánto venía en el sobre.»

«Exigimos el monto **exacto**, ni un wei más. ¿Por qué no aceptar de más y devolver el cambio? Porque devolverle plata a un desconocido es justo la operación delicada del día, y la vamos a ver en treinta segundos. Rechazar es gratis; devolver cambio, no.»

DÍGALO ASÍ · AL ESCRIBIR `saldo[beneficiario] += deposito` EN `devolver`

«Fíjense en lo que **no** acabo de hacer: no le envié el dinero a nadie. Solo lo anoté. El contrato queda debiendo. Eso se llama **patrón de retiro**, y es de las tres o cuatro cosas que hay que saber de memoria.»

DÍGALO ASÍ · AL ESCRIBIR `retirar()`, SEÑALANDO LAS DOS LÍNEAS

«Dos razones para no enviar solo. **Una**: si el que recibe es un contrato que rechaza el envío, se cae mi operación entera. Uno solo bloquearía el sistema para todos. **Dos, y es la grave**: enviarle ether a alguien **le entrega el control de la ejecución**. Su código corre, en medio del mío, y puede volver a entrar y cobrar otra vez. Así se perdieron 3,6 millones de ether en 2016.»

«Y por eso el orden de estas dos líneas no es estilo: **el saldo en cero ANTES de enviar**. Si vuelve a entrar, ya no hay nada que cobrar. Si cambio las dos líneas de orden, este contrato se vacía. Lo vamos a hacer de verdad en la sesión de seguridad.»

Al escribir los eventos: «un evento no cambia el estado: escribe en el registro de la transacción. Cuesta muchísimo menos que guardar lo mismo en una variable, y es de donde una interfaz o una auditoría saca la historia. Con una condición que los sorprende: **un contrato no puede leer sus propios eventos**. Son para afuera. Y acuérdense del paso 3, cuando `registrar` devolvía un `id` que nadie podía ver: esto lo arregla.»

Al escribir `receive` y `fallback`: «estas dos no se llaman nunca a propósito. `receive` corre si alguien manda ether sin decir para qué. `fallback`, si alguien llama a una función que no existe: nombre mal escrito, ABI viejo. Nosotros las dos las rechazamos, y eso es una decisión: plata que entra sin saber de quién ni para qué equipo es plata que después nadie sabe de quién era.»

Demostración en Remix · clic por clic

	CUENTA · FUNCIÓN · ARGUMENTOS	QUÉ DEBE APARECER
1	Cuenta 1 → <code>Deploy</code> con <code>deposito_ = 1000000000000000000</code>	«Diecisiete ceros: son 0,1 ETH en wei. Acá todo se cuenta en wei, que es la unidad más chica. Si alguien pone un cero de menos, el depósito es una décima parte y nadie se da cuenta.» Leer <code>deposito</code> : el mismo número. «Es <code>immutable</code> : se fija al desplegar y no hay función que lo cambie.»
2	Cuenta 1 → <code>registrar</code> <code>"Proyector Epson"</code> . Expandir el detalle de la transacción en la terminal de Remix y buscar los logs.	El evento <code>EquipoRegistrado</code> con el <code>id 1</code> y el nombre. «Ahí está el <code>id</code> que en el paso 3 no podíamos ver. Y esa historia no costó una variable de estado.»
3	Cuenta 2 → <code>prestar</code> con <code>1</code> y <code>2</code> , con el campo VALUE en <code>0</code>	REVIERTE con <code>DepositoIncorrecto</code> y dentro los dos números: <code>1000000000000000000</code> esperado, <code>0</code> recibido. «El error no me dice que me equivoqué: me dice en cuánto.»
4	Poner VALUE en <code>1000000000000000000</code> con la unidad en Wei (o <code>0.1</code> con la unidad en Ether) y repetir.	Funciona. En los <i>logs</i> , el evento <code>Prestado</code> con el <code>vence</code> . Mirar el saldo de la cuenta 2 en el panel de <i>Deploy & Run</i> : bajó un poco más de 0,1 ETH (0,1 del depósito, el resto es gas). «Y el gas no vuelve nunca, pase lo que pase.»
5	Cuenta 2 → <code>devolver</code> con <code>1</code> (a tiempo). Después leer <code>saldo</code> con la dirección de la cuenta 2.	<code>1000000000000000000</code> . Y el saldo de la cuenta 2 en el panel no subió. Señale las dos cosas a la vez: «el contrato dice que me debe 0,1 ETH, y mi billetera no cambió. No está roto: es a propósito.» En los <i>logs</i> , <code>Devuelto</code> con <code>aTiempo: true</code> .
6	Cuenta 2 → <code>retirar</code>	Ahora sí: el saldo de la cuenta 2 sube ~0,1 ETH. Leer <code>saldo(cuenta 2)</code> : <code>0</code> .
7	Cuenta 2 → <code>retirar</code> otra vez. Y cuenta 3 → <code>retirar</code> .	Las dos REVIERTEN con <code>SinSaldo</code> . «Nadie cobra dos veces y nadie cobra lo que no le deben. Dos clics, y es toda la demostración que necesita esa línea.»
8	Mandarle 1 wei al contrato sin llamar ninguna función : en la instancia desplegada, el campo de interacción de bajo nivel (el que recibe <i>calldata</i>) vacío , con VALUE en <code>1</code> Wei, y enviar.	REVIERTE con <code>EnvioDirectoNoPermitido</code> : entró por <code>receive</code> . «Ese ether no tenía dueño ni equipo. Lo devolvemos.»

SI NO ENCUENTRA DÓNDE SE VE EL SALDO · ALTERNATIVA QUE SIEMPRE FUNCIONA

La ubicación exacta del saldo de la cuenta y del saldo de la instancia desplegada cambia entre versiones de Remix. Si no lo ve de una, no pierda tiempo buscándolo: el argumento se sostiene completo con `saldo(dirección)`, que es una función del contrato, más el hecho de que la cuenta 2 pudo pagar el depósito la primera vez y no la segunda. La frase que importa es «el contrato me debe y todavía no me pagó», y eso lo prueba `saldo()`.

LA ROTURA DEL PASO 6 · LA ÚNICA QUE NO SE ARREGLA HOY

Llegó al final del temario y el contrato sigue teniendo un hueco grande. Dígalo, porque es honesto y porque abre el cierre:

«Miren lo que todavía está mal. El que devuelve **un minuto tarde** pierde los 0,1 ETH completos, igual que el que devuelve tres semanas tarde. Y escribimos `diasDeMora` en el paso 5, que cuenta los días, y en el paso 6 **no la usamos para nada**. La regla que programamos no es la regla que queríamos. Ese es el primer ejercicio de la guía, y no tiene solución escrita en ninguna parte.»

Pregunta al grupo

Se pregunta: «Devolví el equipo a tiempo, `saldo` dice que tengo 0,1 ETH y mi billetera no cambió. ¿El contrato está roto?»

Respuesta esperada: no; está anotado a propósito y hay que ir a cobrarlo con `retirar()`. **Y la segunda mitad, que es la que vale:** ¿por qué? Porque enviar ether le entrega el control al que recibe, y porque un destinatario que rechace el envío podría hacer fallar la operación de todos. **Si contestan «para ahorrar gas»:** no es la razón principal, pero no está mal — en efecto quien retira paga su propio gas, y eso también es una decisión de diseño.

SI VA SIN TIEMPO Salte la fila 8 (`receive`) y escriba las dos funciones diciendo en una línea qué hacen. Salte también el segundo `retirar` de la fila 7, quedándose con el de la cuenta 3. Lo que **no** se recorta: las filas 3 a 6. El viaje depósito → anotado → cobrado es el paso 6 completo; todo lo demás es adorno.

10 Si algo falla en vivo

LEER ANTES, NO DURANTE

Programar en vivo se cae. Que se caiga no es el problema; improvisar sí. Estas son las salidas, decididas de antemano.

LO QUE PASA	QUÉ SE HACE, EN ESE ORDEN	QUÉ SE DICE MIENTRAS
No compila	1. Lea el error en voz alta, completo, incluido el número de línea. 2. Pregúnteles qué falta antes de mirar usted. 3. Si en 60 segundos no sale, abra el <code>.sol</code> del paso y compare el bloque, no el archivo. 4. Si en 90 segundos no sale, pegue el archivo completo del paso y siga.	«Este es el trabajo de verdad: el 80 % del tiempo de un programador es leer un error. Vamos a leerlo juntos.» Un error en vivo bien leído enseña más que el paso que venía.
El botón <i>Deploy</i> está apagado y no pasa nada al oprimirlo	1. Mire el campo <i>Value</i>: tiene que estar en 0. Si trae cualquier otro número, Remix no deja desplegar, porque el constructor no es <code>payable</code>. 2. Confirme la marca verde del compilador. 3. Confirme que el desplegable de contratos muestra <code>PrestamoEquipos</code>.	Si pasa en vivo, úselo: «el contrato no acepta dinero porque nadie escribió <code>payable</code> . Rechazar plata que no se pidió también es una regla, y la estamos viendo funcionar antes de escribirla.» <i>Value</i> solo sube de cero en el paso 6.
«No compila» y no se ve ningún error	1. El número rojo sobre el ícono <i>Compile</i> y en la pestaña del archivo es la cantidad de problemas. 2. Alt+F8 salta al problema y muestra el mensaje; también sale al pasar el cursor por el subrayado rojo. 3. En el panel del compilador, los errores salen en tarjetas debajo de los botones: hay que desplazarse. 4. La causa más probable: el compilador no coincide con el <code>pragma</code>.	« <code>ParserError: Source file requires different compiler version</code> no es un error del código: es que Remix abrió con una versión más nueva. Se arregla en el desplegable <i>COMPILER</i> , eligiendo 0.8.28+commit.7893614a , y nunca cambiando el <code>pragma</code> del archivo: esa versión está fija a propósito.»
Remix no despliega, o el botón no responde	1. Confirme que el compilador está en verde: sin compilar no hay nada que desplegar. 2. Confirme que el contrato seleccionado para desplegar es <code>PrestamoEquipos</code>. 3. Confirme <i>ENVIRONMENT</i> en Remix VM y que <i>Account</i> lista las diez cuentas. 4. Recargue la pestaña (sabiendo que puede perder el estado) y vuelva a compilar. 5. Plan B: proyecte las pruebas (§8, plan B) y siga explicando con eso.	«Mientras arranca: ¿qué creen que debería pasar cuando llame a esta función?» Nunca se arregla en silencio. Si el arreglo pasa de dos minutos, se cambia de plan, no se insiste.
Se perdió el estado: el contrato ya no está o el inventario quedó vacío	1. No lo lamente: aprovéchelo. 2. Vuelva a desplegar y a registrar un equipo (dos transacciones, 30 segundos). 3. Use el <code>id</code> que anotó en el tablero (§1, punto 8 y la caja de la pausa).	«Esto es importante y no lo tenía planeado: cada despliegue es un contrato nuevo y vacío . El de hace un rato sigue existiendo con su estado y su dirección; lo que se perdió es mi referencia a él. En una cadena real no se pierde nunca, y eso es justo el problema: tampoco se borra.»

Un estudiante pregunta algo del paso 6 en el paso 2

1. Dígale que acertó el orden del curso, con esas palabras. 2. Escriba la pregunta en una esquina del tablero con su nombre al lado. 3. Diga en qué paso se responde. 4. Cóbrense la deuda: cuando llegue a ese paso, **empiécelo con su pregunta**.

«Esa es exactamente la pregunta del paso 6 y la hizo en el paso 2. La anoto con su nombre y la contesto allá, porque si la contesto ahora tengo que explicarle `payable` sin que hayamos visto por qué hace falta, y eso es lo que me salió mal la clase pasada.» Nunca se despacha con «eso viene después» a secas.

La red local no conecta con Remix (plan A del paso 5)

1. Un intento: confirme que `npx hardhat node` sigue vivo y que el entorno elegido es el de Hardhat. 2. Si no entra de una, **abandone** y pase al plan B (las 25 pruebas). No depure la red delante de la clase.

«No voy a depurar la red local con ustedes mirando: es tiempo de ustedes. Lo mismo se demuestra con las pruebas, y mejor, porque queda escrito.»

Se cae internet o Remix no abre

Toda la clase se puede dar con el proyecto de Hardhat local: `npx hardhat test test/demo/PrestamoEquipos.test.js` más los seis `.sol` abiertos en el editor. Se pierde el clic, no el argumento: las roturas se cuentan y se muestran en el nombre de las pruebas.

«Cambio de herramienta, no de clase. Y fíjense en algo: todo lo que les iba a mostrar a mano está escrito acá y se vuelve a verificar solo. Eso es lo que van a entregar ustedes en el proyecto.»

Un estudiante dice «eso no es así» o trae una fuente que lo contradice

1. Pídale la fuente y ábrala en pantalla. 2. Si tiene razón, dígalo así, sin matizar, y corrija el material. 3. Si no se puede resolver en dos minutos, anótelos como pendiente y respóndalo por el canal del curso.

«Verifiquemos, que es lo que enseña este curso.» Esto no debilita la clase: es la clase.

11 Cierre

MIN 160 - 180

Primero, el recorrido de las seis roturas · 6 minutos

Proyecte esta tabla o dibújela. Es la clase entera en una pantalla, y es lo único que tienen que poder reconstruir.

PASO	LO QUE SE ROMPIÓ	LO QUE ENTRÓ PARA ARREGLARLO
1	Cualquiera se llevaba un equipo que ya estaba prestado, y la transacción salía bien.	<code>if (...) revert</code> con errores personalizados; <code>view</code> para preguntar gratis.
2	El contrato manejaba un equipo. El laboratorio tiene cuarenta.	<code>struct</code> , <code>mapping</code> , un contador a mano, <code>calldata</code> , y el campo <code>existe</code> .
3	Un estudiante cualquiera registraba inventario inventado.	<code>constructor</code> , <code>modifier</code> , una dirección con poder. Y la pregunta de qué pasa si esa llave se pierde.
4	Nadie decía hasta cuándo. Un equipo podía quedar prestado para siempre.	<code>block.timestamp</code> , un plazo con máximo, <code>estaVencido</code> , <code>diasDeMora</code> . Y el reloj que solo se mueve en una red propia.
5	Estar en mora no costaba nada. El contrato sabía y no podía hacer nada.	<code>payable</code> , un depósito exacto, el patrón de retiro, eventos, <code>receive / fallback</code> .

6 El que devuelve un minuto tarde pierde igual que el que devuelve tres semanas tarde. Y `diasDeMora` quedó sin usar.

Nada. Queda abierto. Es el primer ejercicio de la guía y no tiene solución publicada.

Después, la pregunta del curso · 10 minutos

Pregúntela y **espere**. No la conteste en los primeros treinta segundos. Deje que alguien defienda el sí.

DÍGALO ASÍ · LA RESPUESTA HONESTA, MIN 172

«¿Esto necesitaba una blockchain? Para el laboratorio de nuestra universidad, **no**. Una hoja de cálculo y una persona honesta bastan, y salen gratis. Acabamos de escribir 190 líneas sin auditar para hacer algo que hace una hoja de cálculo, y además le pusimos un dueño único cuya llave perdida congela el inventario para siempre.»

«Tendría sentido en dos casos, y solo en dos. **Uno**: si el inventario fuera compartido entre varias universidades que **no se fían entre sí**, y ninguna aceptara que la base de datos la administre otra. **Dos**: si el depósito tuviera que devolverse **sin que nadie pueda decidir no devolverlo**: ni el laboratorio, ni el rector, ni yo.»

«Saber distinguir esos dos casos del resto vale más que saber escribir el contrato. Lo segundo lo hace cualquiera con la documentación abierta. Lo primero es lo que les van a pagar.»

Si alguien defiende el sí con «pero es transparente / es inmutable», no lo corrija: pídale precisión.

«¿Transparente para quién, que hoy no pueda ver la hoja de cálculo? ¿Y qué hacemos cuando registremos mal un equipo, si es inmutable?» La inmutabilidad como argumento a favor, mirada de frente, casi siempre se da vuelta.

Y para terminar · 4 minutos

- **Lo que NO hicimos hoy, dicho explícitamente:** este contrato no está auditado, no tiene pruebas escritas por ustedes, no tiene forma de cambiar de encargado, y nunca tocó dinero real. Las tres cosas tienen sesión propia.
- **Trabajo autónomo:** repetir los seis pasos solos con la guía del estudiante, desde el archivo vacío. No copiando y pegando: escribiendo. Y de los tres ejercicios de ampliación de la guía, intentar el primero (cobrar la mora por día) hasta que compile. No se pide que funcione bien: se pide que lleguen a la pregunta de diseño que trae adentro.
- **Lo que viene:** este mismo código, pero usando `Ownable` de OpenZeppelin en lugar del modificador escrito a mano, y pruebas automáticas escritas por ustedes.
- **La última frase, si alcanza:** «hoy escribimos cinco versiones malas a propósito. En su proyecto van a escribir varias sin querer. La diferencia va a ser si las rompen ustedes antes de entregarlas o si las rompe otro después.»

12 Datos verificados y lo que hay que confirmar antes de dictar

VERACIDAD

DATO	ESTADO	DETALLE
Los seis <code>.sol</code> compilan con 0.8.28 sin avisos	VERIFICADO	Comprobado por el docente antes de preparar este guion.
25 pruebas en verde en <code>test/demo/PrestamoEquipos.test.js</code>	VERIFICADO	Incluyen mora, división entera truncada y el borde del último segundo del plazo.

<code>adelantar-reloj.js</code> mueve el reloj	VERIFICADO	Con <code>npx hardhat node</code> corriendo, <code>DIAS=3</code> avanzó exactamente 3,00 días.
Red local: chainId 31337, RPC <code>http://127.0.0.1:8545</code>	VERIFICADO	Valores por omisión del nodo de Hardhat del proyecto.
Remix VM: 10 cuentas con 100 ETH; el reloj no se puede adelantar	VERIFICADO	Documentación de Remix, <code>run.html</code> . Las cuentas se confirman en el selector <i>Account</i> al entrar.
Botones: azul = <code>view</code> , naranja = transacción, rojo = <code>payable</code>	VERIFICADO	Documentación de Remix, <code>create_deploy.html</code> ; el texto advierte que depende del tema visual . Si su tema los pinta distinto, diga «los de un color son gratis y los del otro no».
<code>PLAZO_MAXIMO</code> = 1209600 segundos	VERIFICADO	14 × 86 400. Calculado, no citado.
Nombre exacto del entorno de Hardhat en el desplegable	⚠ VERIFICAR ANTES DE DICTAR	La documentación lo llama Hardhat Provider , dentro de los «Development Environments»; la interfaz lo ha mostrado como <i>Dev - Hardhat Provider</i> . Ábralo en el montaje y use el nombre que vea . Si no coincide, dígalo genérico: «el entorno que conecta con un nodo local».
Nombre exacto del <i>fork</i> de la Remix VM (¿Cancun?)	⚠ VERIFICAR ANTES DE DICTAR	La documentación dice que hay varias Remix VM, cada una con su <i>hard fork</i> entre paréntesis, y que se elija la primera del desplegable ; no fija cuál es. No afirme un nombre: diga «la primera Remix VM de la lista».
Texto literal con que Remix muestra un error personalizado	⚠ VERIFICAR ANTES DE DICTAR	Este guion solo afirma que la terminal marca la transacción como fallida y muestra el nombre del error con sus argumentos . Provoque un <code>EquipoOcupado</code> en el montaje y vea cómo se presenta: va a leerlo en voz alta seis veces.
Dónde se ve el saldo de una cuenta y de la instancia desplegada	⚠ VERIFICAR ANTES DE DICTAR	Cambia de sitio entre versiones. Si no lo encuentra, use la alternativa de la caja del paso 6: <code>saldo(dirección)</code> sostiene el argumento completo.
Campo de interacción de bajo nivel para mandar 1 wei sin llamar función	⚠ VERIFICAR ANTES DE DICTAR	Existe en el panel de la instancia desplegada, pero el nombre del campo y del botón varía. Pruébalo en el montaje; si no lo encuentra en un minuto, salte la fila 8 del paso 6 (ya está marcada como recortable).

DÓNDE ESTÁ TODO

Las seis versiones: `material/clase-solidity-paso-a-paso/pasos/paso-1.sol ... paso-6.sol` . Copia probada de la versión final: `material/laboratorios-evm/contracts/demo/PrestamoEquipos.sol` .

Pruebas: `material/laboratorios-evm/test/demo/PrestamoEquipos.test.js` . Reloj: `material/laboratorios-evm/scripts/demo/adelantar-reloj.js` . Guía del estudiante: `guia-estudiante.pdf` , en esta misma carpeta.

Documentación de Remix consultada para este guion: `https://remix-ide.readthedocs.io/en/latest/run.html` y `https://remix-ide.readthedocs.io/en/latest/create_deploy.html` .